

PERSONAL WORK REPORT · WEEK 3

My work on the Pre-Thesis Demo

Mr. Kongkidakorn Chuayjang · Head Programmer

REPORTING WEEK

Week 3 ·

PROJECT

Pre-Thesis Demo · Unity 6.5 + Mirror + EOS

Every claim in this deck is traceable to a commit in the team repository.

What is mine, and what is not

This project has two programmers. This slide draws the line so nothing later is ambiguous.

MY WORK

Written by me

- Save system (encrypted)
- Room creation & matchmaking
- Room browser data layer
- Offline / online mode switch
- Scene-persistence utilities
- Editor tooling
- Persistent HUD root

THEIRS, REWRITTEN BY ME

Shared ownership

Atiroj designed and wrote these as single-player systems. I rewrote them to run under server authority for six players.

- Interaction system
- Health & damage
- Player movement
- First-person camera
- Health bar UI

ATIROJ'S WORK

Not mine

Written by Atiroj and merged into my branch. I am not claiming any of it.

- Original single-player design of the systems in the middle column
- Downed / revive feature
- Gameplay prop setup in test scenes

Evidence for this split: my own commits versus the three merges that brought Atiroj's work in.

Encrypted save system

What it does

- Writes progression as AES-256 encrypted JSON — not readable or editable in a text editor.
- Loads automatically before the first scene, so no system has to ask whether a save exists.
- One static source of truth — no component keeps its own copy to drift out of sync.
- Permanent items capped at one copy, removed on drop, trade or death.
- Custom Inspector shows live save contents during Play, so progression bugs are visible without a debugger.

A decision I want to defend

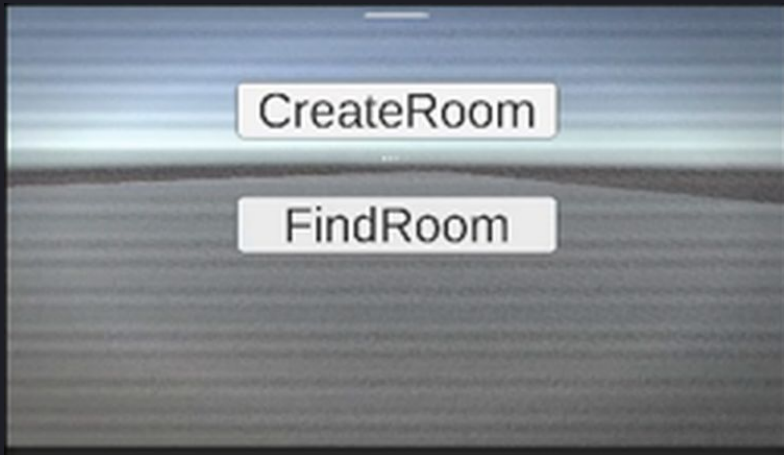
The encryption key is not stored as one string. It is split into four byte arrays and assembled at runtime.

```
// no single string to grep  
// out of the binary  
keyPart1 ... keyPart4 → GetKey()
```

A determined attacker with the binary still wins — true of any client-side save. The achievable goal is narrower: make casual save editing not worth the effort, which is what damages a co-op economy.

347 lines · 4 files

Room creation & matchmaking



From my playtest recording

Create Room opens a lobby on Epic's service and starts hosting. Find Room searches and returns every open room.

1

A room is a data contract, not a button

Map, difficulty, player limit and privacy live in one validated structure. The UI cannot create a room the join path can't read.

2

The browser knows before you join

Each room advertises map, difficulty, lock state and a live 2/6 player count — so a player picks a room instead of gambling.

3

Passwords are never a room attribute

I read Epic's lobby API: attributes marked private are still readable by anyone searching. So the password is verified after joining.

450 lines · 5 files

Making Atiroj's systems multiplayer-safe

Atiroj wrote the interaction and health systems as single-player code, which is the right way to write them. My job was the rewrite that makes them survive six players and a hostile client.

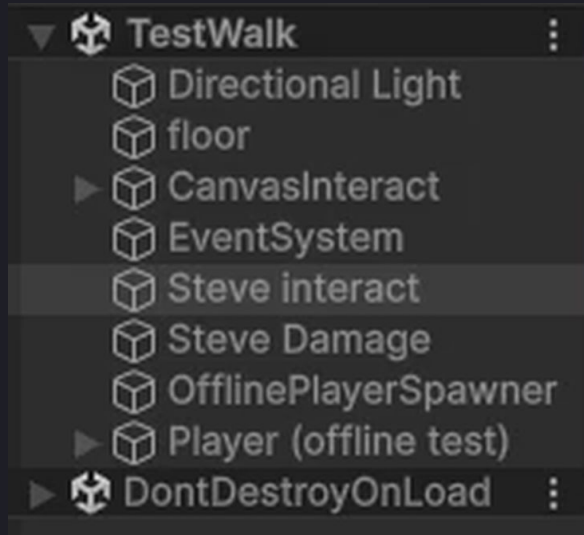
BEFORE — single-player

AFTER — server-authoritative

Interaction	Pressing E ran the door's code directly on the machine that pressed it.	The client asks the server which object it wants. The server re-checks distance before opening anything.
Health & damage	A collider applied damage locally, so each machine saw a different health value.	Only the server applies damage; health syncs outward. Every player sees the same number.
Player control	Every copy of the player prefab read the keyboard — one key press moved all of them.	Only the locally owned player reads input. Remote copies are driven by the network.
Late joiners	A player joining after a door opened saw it closed.	State is re-applied on connect, so the world is consistent mid-run.

Credit where it is due: the features are Atiroj's. The networking model they now run on is mine.

Letting the team test without hosting a match



TestWalk in Play mode — props still present

The problem

Mirror switches off every scene object carrying a network identity so the server controls when it appears. Correct in a match — and it means a teammate pressing Play in a test scene watches every prop vanish.

What I built

- One offline check every system consults, so gameplay code is written once and behaves correctly with or without a server.
- A spawner that places a test player and restores the objects Mirror disabled — and does nothing once a match is running.
- Persistence helpers so the Epic manager and HUD survive scene changes instead of being destroyed mid-match.

Honest note: this took three attempts. The first two assumed Mirror disables objects before Awake — it does not.

Playtest — two machines, one world

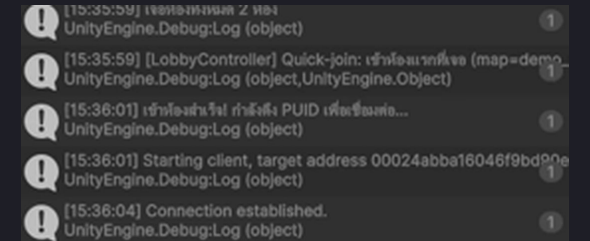


Two Player(Clone) in one scene

Host and joined client both present.



The other player, seen from my client



Connection established

The client resolved the host's Epic user ID and connected through it — no IP address involved.

Recordings supplied with this deck

- 13-48-04.mp4 (0:54) — offline sandbox: interactables stay present, damage trigger fires
- 15-35-02.mp4 (1:18) — creating a room on Epic and starting the host
- 15-35-42.mp4 (0:40) — second client finds the room, joins, both players appear

Bugs I found by reading, not guessing

Each had a plausible wrong explanation that would have cost days. I found the real cause in the library's own source.

	SYMPTOM	REAL CAUSE	WHAT I DID
1	Build would not compile	Two Epic SDKs installed at once, colliding on the same native library.	Removed the unused one. 0 errors, 0 warnings.
2	Client joined an empty world	A server can spawn a prefab it references; a client can only spawn one that is registered.	Identified the checkout mismatch.
3	Props disappeared on Play	Mirror disables scene objects after Awake, so an Awake-time fix runs too early.	Rewrote the fix to watch across frames.
4	Rooms rejected with InvalidUser	Epic login is asynchronous; the button could be pressed before login finished.	Gated room creation on login state.
5	Damage did nothing	Detection relied on a tag the spawned prefab did not carry.	Detect by component instead of tag.

What I am doing next

1

Room browser UI

The search already returns every open room with map, difficulty, lock state and player count. Nothing draws it yet — that is my next build.

2

Move the match into a real scene

The online test currently runs in the menu scene. Next is a proper level load with players spawning into it.

3

Private-room passwords

Design is settled — verified after joining, never advertised. Implementation is mine.

4

Start the 13 Rules system

The design thesis of the game, still unstated. I want it under way before it becomes the thing we ran out of time for.

I will report against this list in next week's personal presentation.

Everything I have built so far

Written from scratch

- AES-256 encrypted save system with auto-load
- Permanent items capped at 1, lost on drop or death
- Custom Inspector for live save contents
- Room creation on EOS → Mirror host, and room search
- Room data contract: map, difficulty, limit, privacy
- Offline / online mode switch + scene persistence
- Editor tooling for network setup checks

Rewritten for multiplayer

- Interaction → client asks, server validates range
- Health & damage → server-only, synced outward
- Player control → only the owned player reads input
- Camera → remote cameras and listeners disabled
- Late joiners → state re-applied on connect
- Health bar UI → binds to the local player only

Verified working

- Two machines in one session over Epic
- Both players spawn, move and see each other
- Room created with full config carried across
- Solo sandbox scenes run with no server
- Build compiles with 0 errors and 0 warnings

1,360

lines written from scratch

1,461

lines rewritten for multiplayer

11

commits of my own

5

hard bugs root-caused